Insertion and Deletion of a node using the Doubly Circular Linked List

1. Insertion of a node at the begining

main.c | Share | Run | Output

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void insertbegin(){
    struct node *newnode,*last;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 80;
    if(head == NULL){
        newnode->next = newnode;
        newnode->prev = newnode;
        head = newnode;
        return;
    }
    last = head->prev;
    newnode->next = head;
    newnode->prev = last;
    last->next = newnode;
    head->prev = newnode;
```

```
List: 80 20 40 60

=== Code Execution Successful ===
```

main.c | Share | Run | Output | Clear

```c
    head = newnode;
}
void create(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 20;
    second->data = 40;
    third->data = 60;
    head = first;
    first->next = second;
    first->prev = third;
    second->next = third;
    second->prev = first;
    third->next = first;
    third->prev = second;
}
void display(){
    struct node *temp = head;
    printf("%d ",temp->data);
    temp = temp->next;
    while(temp != head){
```

```
List: 80 20 40 60

=== Code Execution Successful ===
```

```c
    while(temp != head){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main(){
    create();
    insertbegin();
    printf("List: ");
    display();
    return 0;
}
```

2. Insertion of a node at the Ending

main.c

```c
    last->next = head;
    head->prev = last;
}
void insertEnd(int data)
{
    struct node *newnode, *last;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;
    last = head->prev;
    newnode->next = head;
    newnode->prev = last;
    last->next = newnode;
    head->prev = newnode;
}
void display()
{
    struct node *temp = head;
    if(head == NULL)
    {
        printf("List is empty");
        return;
    }
    printf("%d ", temp->data);
```

Output

```
After inserting at end: 10 30 100 90

=== Code Execution Successful ===
```

```c
    }
    printf("%d ", temp->data);
    temp = temp->next;
    while(temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    beginning();
    insertEnd(90);
    printf("After inserting at end: ");
    display();
    return 0;
}
```

3. Deletion of a Node at the begining

main.c

```c
    last->next = head;
    head->prev = last;
}
void deleteBeginning()
{
    struct node *temp,*last;
    if(head == NULL)
    {
        printf("List is empty");
        return;
    }
    temp = head;
    last = head->prev;
    head = head->next;
    head->prev = last;
    last->next = head;
    free(temp);
}
void display()
{
    struct node *temp = head;
    if(head == NULL)
    {
        printf("List is empty");
        return;
    }
    printf("%d ", temp->data);
    temp = temp->next;
    while(temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    beginning();
    deleteBeginning();
    printf("After deleting at beginning: ");
    display();
    return 0;
}
```

Output

```
After deleting at beginning: 67 80

=== Code Execution Successful ===
```

4. Deletion of a node at the end of a linked list

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct node
{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning()
{
    struct node *first,*second,*last;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    last = (struct node*)malloc(sizeof(struct node));
    first->data = 60;
    second->data = 80;
    last->data = 90;
    head = first;
    first->next = second;
    second->prev = first;
    second->next = last;
    last->prev = second;
```

Output

```
After deleting at end: 60 80

=== Code Execution Successful ===
```

main.c

Share Run

```c
    last->prev = second;
    last->next = head;
    head->prev = last;
}
void deleteEnd()
{
    struct node *temp,*last;
    if(head == NULL)
    {
        printf("List is empty");
        return;
    }
    last = head->prev;
    temp = last;
    last->prev->next = head;
    head->prev = last->prev;
    free(temp);
}
void display()
{
    struct node *temp = head;
    if(head == NULL)
    {
```

Output

```
After deleting at end: 60 80

=== Code Execution Successful ===
```

```c
    if(head == NULL)
    {
        printf("List is empty");
        return;
    }
    printf("%d ", temp->data);
    temp = temp->next;
    while(temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main()
{
    beginning();
    deleteEnd();
    printf("After deleting at end: ");
    display();
    return 0;
}
```